POKROČILÉ PROGRAMOVÁNÍ (PPRO) • FIM UHK

Ordinace Na Vyhlídce

Objednávací a evidenční systém pro praktické lékaře

Dokumentace semestrálního projektu

Zadání	A – ordinace praktických lékařů
Autor	Jakub Silný
Studijní obor	Aplikovaná informatika
Akademický rok	2026/2027, zimní semestr
Repozitář	https://github.com/JakubSilny/ppro-ordinace

Univerzita Hradec Králové, Fakulta informatiky a managementu

Obsah

1 Úvod 2

2 Datový model 3

3 Architektura 5

4 Uživatelské rozhraní 6

5 Použité technologie 9

6 Spuštění 9

7 Testy 10

8 Protokol o zacyklení agenta 11

9 Přiznání práce s AI 12

1 Úvod

Ordinace Na Vyhlídce je soukromá praxe se třemi lékaři na částečný úvazek a jednou recepční. Dosud se objednávalo do papírového diáře, volné termíny se hledaly listováním a výkony pro pojišťovnu se na konci měsíce ručně přepisovaly do tabulky. Dvakrát za rok se stalo, že na stejný čas byli objednaní dva pacienti.

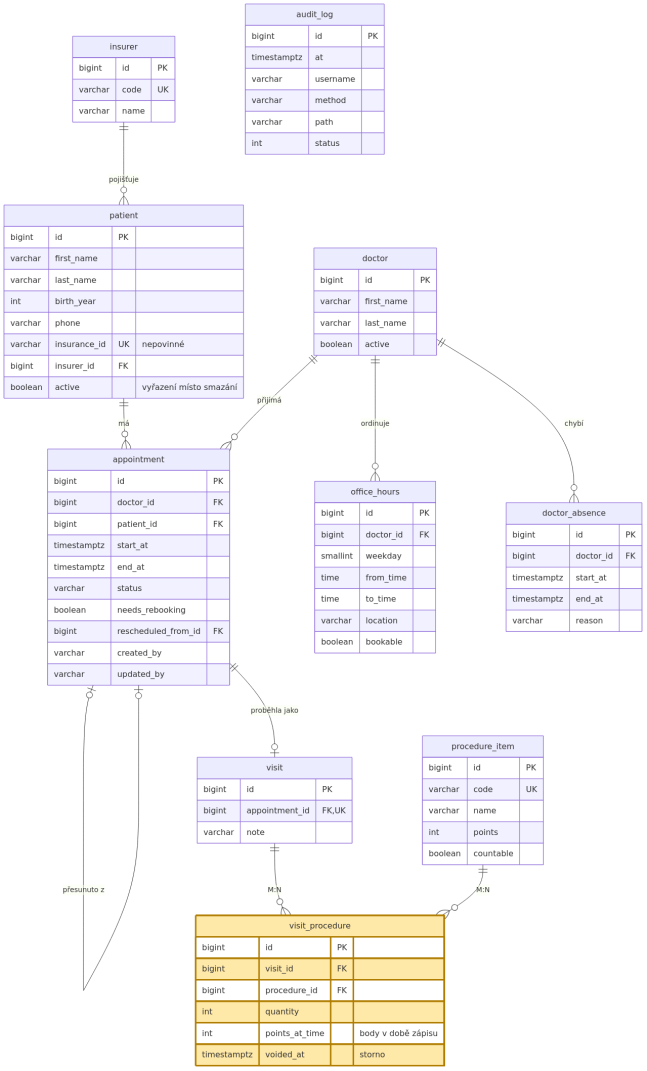
Aplikace tyto činnosti nahrazuje webovým systémem pro recepci:

- týdenní diář lékaře s volnými okny, objednání jedním kliknutím,
- rušení, přesun a evidence nedostavení se, přehled pacientů, kteří ruší opakovaně,
- evidence pacientů s hlídáním duplicit,
- zápis návštěvy a provedených výkonů z číselníku,
- nepřítomnost lékaře a seznam rezervací k přeobjednání,
- měsíční výkaz výkonů a bodů pro pojišťovnu (obrazovka, tisk, CSV).

Klíčová pravidla klienta vynucuje aplikace i databáze: dvě rezervace se nepřekryjí, mimo ordinační hodiny objednat nejde, výkon nelze zapsat před začátkem návštěvy a historie pacienta se nemaže.

2 Datový model

Schéma je ve verzovaných migracích Flyway (`src/main/resources/db/migration`): V1 schéma, V2 číselníky a syntetická data, V3 audit přístupů, V4 audit jen pro zápis a autoři rezervací. Ukázkový provoz pro předvedení přidává migrace V100 jen v profilu `demo` (`src/main/resources/db/demo`).



Obrázek 1: ER diagram. Zvýrazněná tabulka `visit_procedure` je spojovací tabulka vazby M:N mezi návštěvou a výkonem.

Entita	Význam	Klíčové atributy a vazby
`insurer`	číselník zdravotních pojišťoven	`code` (unikátní, např. 111)

Entita	Význam	Klíčové atributy a vazby
`patient`	pacient	`insurance_id` (nepovinné, unikátní), `phone` (jen číslice), `active` (vyřazení místo smazání); N:1 k `insurer`
`doctor`	lékař	`active`; 1:N k ordinačním hodinám, absencím a rezervacím
`office_hours`	týdenní šablona ordinačních hodin	`weekday`, `from_time`–`to_time`, `location`, `bookable` (zda se tam objednává)
`doctor_absence`	nepřítomnost lékaře	časový interval `start_at`–`end_at`, `reason`
`appointment`	rezervace pacienta u lékaře	`start_at`–`end_at`, `status` (BOOKED, COMPLETED, CANCELLED, NO_SHOW, RESCHEDULED), `needs_rebooking`, `rescheduled_from_id` (odkaz na původní rezervaci při přesunu), `created_by`/`updated_by`
`visit`	uskutečněná návštěva	`appointment_id` (unikátní, tedy 1:1 k rezervaci), `note`
`procedure_item`	číselník výkonů	`code` (unikátní), `points`, `countable` (zda se zadává počet)
`visit_procedure`	**vazba M:N** návštěva–výkon	`quantity`, `points_at_time`, `voided_at`
`audit_log`	záznam přístupů k osobním údajům	kdo (`username`), kdy (`at`), co (`method`, `path`), výsledek (`status`); jen pro zápis

Vazba M:N. Na jedné návštěvě proběhne více výkonů a tentýž výkon se opakuje u mnoha návštěv. Spojovací tabulka `visit_procedure` nese vlastní atributy: počet, body zmražené v době zápisu (změna ceníku neovlivní minulé výkazy) a čas storna. V JPA je zmapovaná jako samostatná entita `VisitProcedure`, ne jako `@ManyToMany`, právě kvůli těmto atributům.

Integritní omezení v databázi:

- `EXCLUDE USING gist (doctor_id WITH =, tstzrange(start_at, end_at) WITH`

`&&`) pro aktivní stavy znamená, že lékař nemá dvě překrývající se rezervace. Totéž platí pro pacienta.
- `CHECK (end_at > start_at)` u rezervací, absencí a ordinačních hodin.
- `UNIQUE (insurance_id)`. Číslo pojištěnce je nepovinné, ale nesmí se opakovat.
- Částečný unikátní index `(visit_id, procedure_id) WHERE voided_at IS NULL`: jeden aktivní zápis výkonu na návštěvu.
- `CHECK (quantity >= 1)`, `CHECK (phone ~ '^[0-9]+$')` a další.

3 Architektura

Aplikace je třívrstvá a závislosti vedou jen jedním směrem:

```
prohlížeč (HTML/CSS/JS)  --REST/JSON-->  api (controllery, DTO)
                                            |
                                            v
                                       service (pravidla, transakce)
                                            |
                                            v
                                  repository (Spring Data JPA)  -->  PostgreSQL
```

Kód je rozdělený do balíčků podle funkce (`patient`, `doctor`, `appointment`, `visit`, `audit`, `gdpr`, `security`) a uvnitř nich podle vrstev. Controller neobsahuje SQL ani logiku. Jen převádí požadavek na volání služby a výsledek na DTO (Java `record`).

Jak je směr závislostí hlídaný. Test `LayersTest` (ArchUnit) při každém buildu ověří, že balíčky `api` nesahají na `repository` ani na JPA/JDBC, `service` nezávisí na `api` a `repository` nezávisí na vyšších vrstvách. Porušení shodí `./mvnw verify` i CI. Při zavedení testu se tak našly dva záznamy z dotazů (`CancellationStat`, `ReportLine`), které ležely v `repository` a používal je controller. Přesunuly se do `dto`.

Pravidla ve službě a v databázi. `AppointmentService` před uložením ověří délku, ordinační hodiny, nepřítomnost lékaře a kolize. Databáze tatáž pravidla drží znovu, takže když se dva požadavky na stejný termín potkají současně, uspěje jen jeden. Druhý skončí výjimkou z constraintu, kterou `ApiExceptionHandler` převede na 409. Test `AppointmentConcurrencyIT` to ověřuje dvěma vlákny.

Čas. Služby dostávají `java.time.Clock` (Europe/Prague), takže testy mohou čas zafixovat. Časy se ukládají jako `TIMESTAMPTZ`. Test `sameLocalTimeIsAcceptedBeforeAndAfterDaylightSavingChange` hlídá přechod na zimní čas.

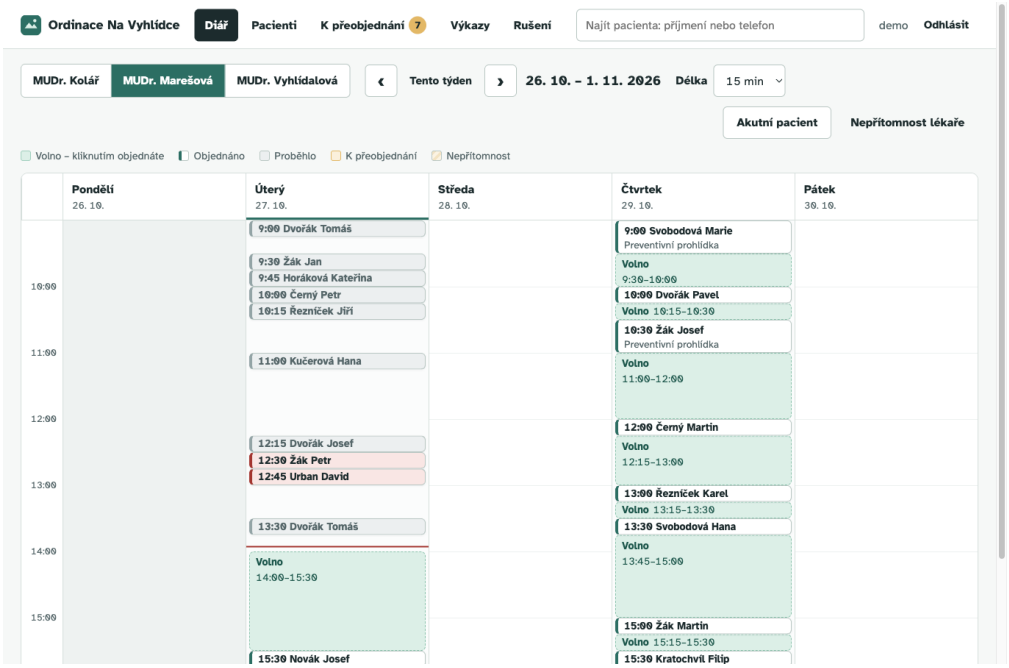
Volné termíny. `TimeRange.free` vezme bookovatelné intervaly týdne a odečte rezervace, nepřítomnosti a uplynulý čas. Vrací souvislá okna, do kterých se vejde zvolená délka.

Chyby. Výjimky se mapují na `application/problem+json`: `NotFoundException` na 404, `ConflictException` na 409 (případně s `existingId` duplicitního pacienta), `BusinessRuleException` na 422. Rozhraní je překládá do češtiny.

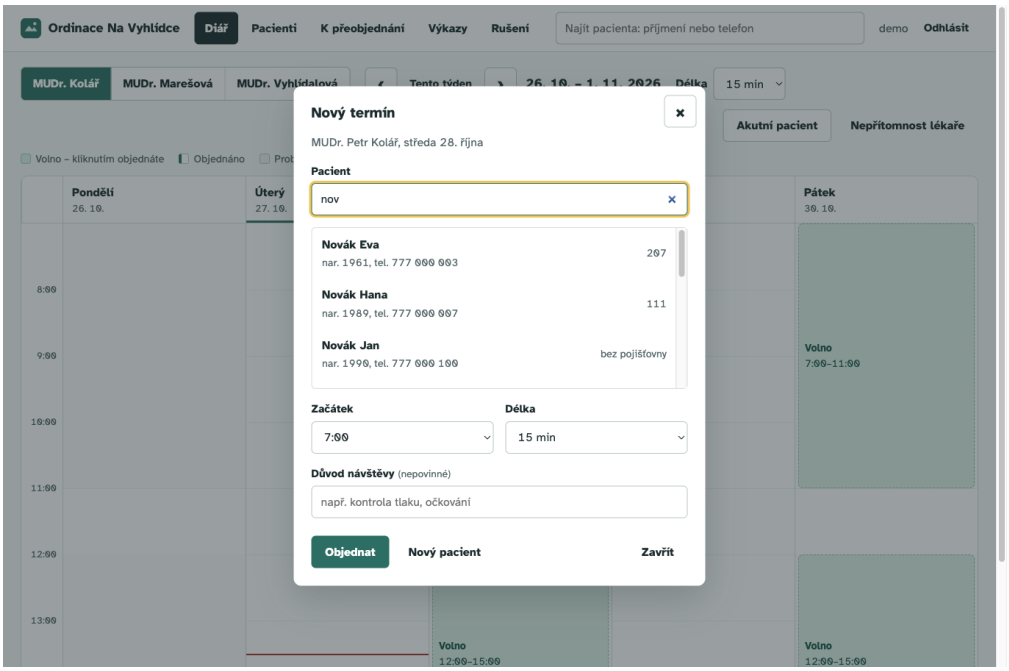
Bezpečnost a GDPR. Spring Security s přihlašovacím formulářem, CSRF tokenem v cookie, Content-Security-Policy bez inline skriptů a vypršením relace po 30 minutách. `AuditInterceptor` zapisuje každé volání API s osobními údaji do `audit_log` a `LoginEvents` tam zapisuje i každý pokus o přihlášení. Tabulka `audit_log` jde jen doplňovat: úpravu, smazání i `TRUNCATE` odmítne trigger v databázi (migrace V4). U rezervace se ukládá, kdo ji založil a kdo ji naposledy změnil (`@CreatedBy`/`@LastModifiedBy` ze Spring Data, hodnotu dodá přihlášený uživatel). Po pěti špatných heslech se jméno na 5 minut zablokuje (`LoginAttempts`). Zamyká se i neexistující jméno, aby se podle hlášky nedalo poznat, které účty existují. Hesla v `APP_USERS` mohou být uložená jako bcrypt hash. Kontejner aplikace běží pod neprivilegovaným uživatelem, s jen pro čtení souborovým systémem, bez linuxových oprávnění (`cap_drop: ALL`) a s `no-new-privileges`. Export údajů pacienta (`/api/patients/{id}/export`) slouží pro žádosti podle čl. 15 a 20 GDPR.

4 Uživatelské rozhraní

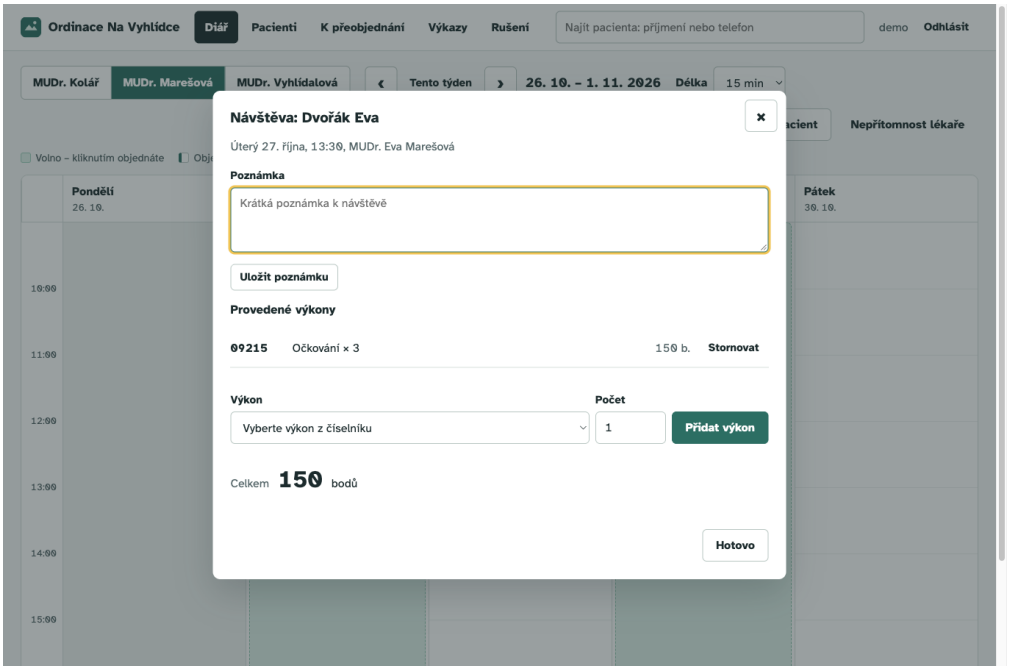
Rozhraní je určené recepční, která „s počítačem moc nekamarádí“: velké ovládací prvky (min. 44 px), písmo Atkinson Hyperlegible navržené pro snadné čtení, hledání pacienta vždy po ruce v horní liště a srozumitelné české hlášky místo technických chyb.



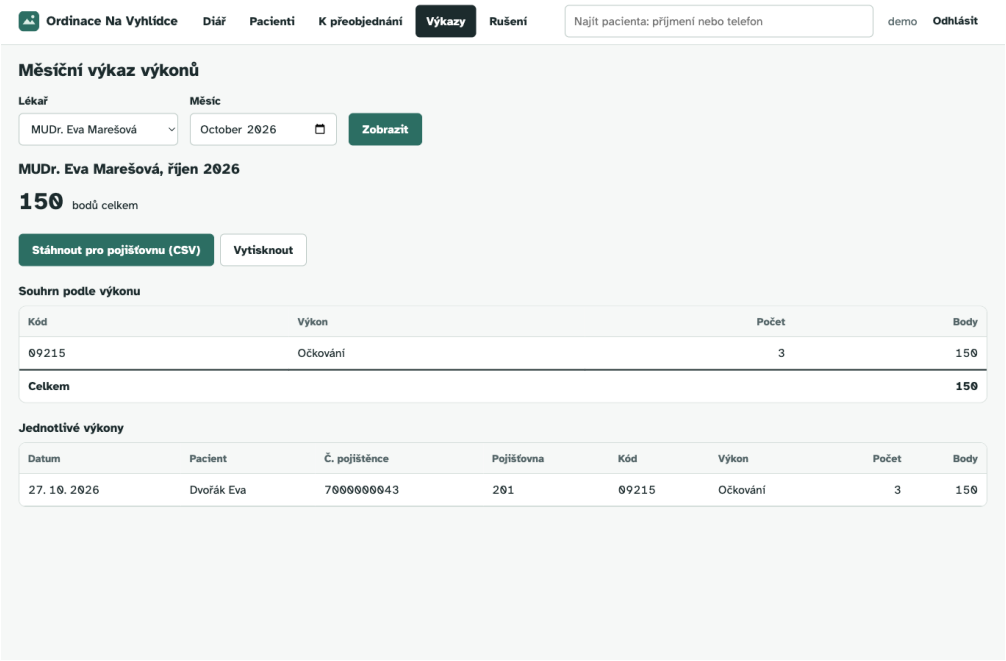
Obrázek 2: Týdenní diář lékaře s volnými okny a objednaným pacientem



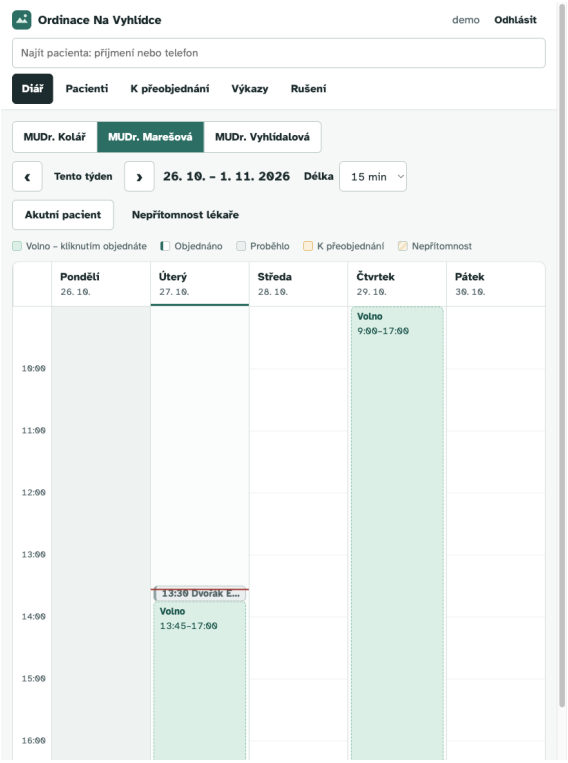
Obrázek 3: Objednání z volného okna: hledání pacienta, začátek a délka termínu



Obrázek 4: Zápis návštěvy a výkonů, body se sčítají průběžně



Obrázek 5: Měsíční výkaz pro pojišťovnu se souhrnem a exportem do CSV



Obrázek 6: Diář na tabletu

5 Použité technologie

Technologie	Proč
Java 21, Spring Boot 3.5	Javu znám z předchozích předmětů; records pro DTO, konstruktorová injekce a hotová podpora REST, JPA, validace i testů
Spring Data JPA, Hibernate	repozitáře bez ručního SQL tam, kde stačí odvozené dotazy
PostgreSQL 16	exclusion constraint nad časovými intervaly (`btree_gist`), `unaccent` pro hledání bez diakritiky
Flyway	verzované migrace, stejné schéma v Dockeru i v testech
Spring Security	přihlášení, CSRF, bezpečnostní hlavičky
HTML, CSS, JavaScript (bez frameworku)	jedna stránka bez buildu, rychlé načtení na recepčním počítači
JUnit 5, AssertJ, Mockito, Testcontainers	jednotkové a integrační testy proti skutečné PostgreSQL
ArchUnit	test, který hlídá směr závislostí mezi vrstvami
JaCoCo	měření pokrytí kódu testy, report v CI
Playwright	prohlížečové testy hlavních scénářů (desktop i tablet) a nahraná prohlídka pro klienta
Docker Compose, GitHub Actions	spuštění jedním příkazem, CI při každém pushi
gitleaks, Grype	v CI hledají hesla v historii repozitáře a známé zranitelnosti v Docker image

6 Spuštění

Prerekvizity: Docker Desktop (nebo Docker Engine s pluginem Docker Compose) a Git. Java ani Maven nejsou potřeba, aplikace se sestaví uvnitř Dockeru. Volné porty 8080 a 5432.

Postup:

1. Stáhnout repozitář: `git clone https://github.com/JakubSilny/ppro-ordinace.git` a přejít do složky `ppro-ordinace`.
2. Vytvořit soubor s nastavením z ukázky: `cp .env.example .env`. Pro vyzkoušení ho není nutné měnit. Pro ostrý provoz se v něm nastaví vlastní heslo databáze (`DB_PASSWORD`) a účty personálu (`APP_USERS`, formát `jmeno:heslo,jmeno2:heslo2`).
3. Spustit aplikaci a databázi: `docker compose up --build -d`. První sestavení trvá několik minut.
4. Ověřit, že běží: `docker compose ps` ukáže kontejnery `app` a `db` ve stavu *running (healthy)*.
5. Otevřít v prohlížeči `http://localhost:8080` a přihlásit se.

Adresy služeb:

Služba	Adresa
webová aplikace a REST API	`http://localhost:8080` (API pod `/api/…`)
PostgreSQL	`127.0.0.1:5432`, databáze, uživatel a heslo podle `.env` (výchozí `ordinace` / `ordinace` / `zmente-heslo-databaze`)

Přihlašovací údaje k testovacím datům: s nezměněným `.env.example` jméno **`recepce`** a heslo **`zmente-toto-heslo`**. Výchozí profil `demo` založí ukázkový provoz: 130 syntetických pacientů, 8 týdnů proběhlých návštěv s výkony, objednané termíny na 2 týdny dopředu a nepřítomnost lékaře s rezervacemi k přeobjednání. Všechna data jsou smyšlená.

Zastavení: `docker compose down` (data zůstanou), `docker compose down -v` (smaže i databázi; při dalším startu se založí znovu).

7 Testy

- **Jednotkové testy** (`*Test`) testují aplikační logiku s mocky na hranicích: `AppointmentServiceTest` (ordinační hodiny, polední pauza, minulost, absence, kolize, přesun, no-show, DST, volné termíny), `TimeRangeTest`, `VisitServiceTest`, `MonthlyReportTest`, `PatientServiceTest`, `DoctorServiceTest`, `PhoneNumbersTest`, `LoginAttemptsTest` (zámek po špatných heslech a jeho vypršení). `LayersTest` (ArchUnit) hlídá vrstvy.
- **Integrační testy** (`*IT`) běží proti PostgreSQL v Testcontainers: překryvy a constrainty přímo v DB (`SchemaConstraintsIT`), API rezervací, souběžné objednání (`AppointmentConcurrencyIT`), návštěva, výkaz a CSV (`VisitApiIT`), přihlášení, zablokování po špatných heslech, CSRF, audit a export (`SecurityIT`), audit, který nejde upravit ani smazat (`SchemaConstraintsIT`).
- **Prohlížečové testy** (`e2e/`, Playwright): přihlášení a odhlášení, objednání a zrušení z diáře, přesun termínu, nový pacient a hlídání duplicity, výkaz, rozpis dne.

Spuštění: `./mvnw verify` (pokrytí pak v `target/site/jacoco/index.html`). Pro E2E viz README. CI v GitHub Actions spouští při každém pushi a pull requestu obojí. Navíc vypíše pokrytí do souhrnu běhu, prohledá historii na hesla (gitleaks), Docker image na opravitelné vysoké a kritické zranitelnosti (Grype) a ověří, že `dokumentace.pdf` není starší než `dokumentace.md`.

Počty a výsledek: 46 jednotkových testů, 29 integračních testů a 8 prohlížečových scénářů, které v CI běží dvakrát (desktop a tablet). Poslední běh CI 27. 10. 2026 na commitu `bb304e6` prošel ve všech jobech: build a testy, kontroly (gitleaks, aktuálnost PDF), sken image a E2E.

8 Protokol o zacyklení agenta

Situace. 27. 10. 2026 vzniklo s agentem (Claude Code) během jednoho dne přes dvacet commitů: rezervace, návštěvy, výkaz, přihlášení, GDPR, webové rozhraní, E2E testy a rozšíření CI. Každý krok agent uzavřel jako hotový. Commity ale zůstaly jen lokálně, na GitHub odešly až večer najednou spolu s dokumentací.

První signál. Po pushi se u pull requestu #2 objevilo „5 failing and 4 successful checks“. Selhal build s testy, kontrola hesel (gitleaks) i sken zranitelností Docker image. Do té doby nic nenaznačovalo problém, protože poslední zelené CI bylo z 23. 10.

Co bylo špatně. Šlo o tři nezávislé chyby, které se do té doby nikde neprojevily:

1. *Integrační testy nikdy neběžely lokálně.* Testcontainers 1.20 ze Spring Bootu 3.4.1 se nedokázal připojit k Dockeru 29 („Could not find a valid Docker environment“), takže všech 29 integračních testů lokálně končilo chybou. CI, kde by se to ukázalo, se nespouštělo, protože commity nebyly pushnuté. Tři testy byly přitom opravdu rozbité: hledání podle části telefonu s předvolbou `+420` vracelo prázdný výsledek (skutečná chyba v aplikaci), test přihlášení neposílal hlavičku `Accept: text/html` jako prohlížeč a test diáře počítal s tím, že v daném dni nejsou rezervace z jiných testů.
2. *Zastaralé závislosti.* Spring Boot 3.4.1 z prosince 2024 obsahoval Tomcat, Jackson, Spring Security a ovladač PostgreSQL se známými kritickými zranitelnostmi. Sken Grype, který agent sám do CI přidal, je správně zastavil.
3. *Chybějící oprávnění.* gitleaks u pull requestu potřebuje číst seznam commitů (`pull-requests: read`), workflow mělo jen `contents: read`. Při pushi to fungovalo, chyba se projevila až u prvního PR.

Jak jsem to rozsekl. Agentovi jsem místo popisu poslal přímo screenshot a výpisy z CI. Agent si logy stáhl přes `gh run view --log-failed`, rozdělil selhání na tři příčiny a pustil `./mvnw verify` lokálně. Tím se ukázal problém s Dockerem. Povýšení na Spring Boot 3.5.16 (s novějším Testcontainers) vyřešilo současně lokální běh i většinu zranitelností. Zbylé knihovny se přepsaly verzí v `pom.xml` a výsledek se ověřil během Grype v Dockeru ještě před pushem. Povýšení ale odhalilo další chybu: nová Spring Security ověřuje heslo i u zamčeného účtu a náhradní účet mělo heslo bez prefixu `{noop}`. Agent při opravě testu diáře napsal nejdřív kontrolu závislou na časovém pásmu (`09:00Z`), kterou sám před spuštěním nahradil kontrolou podle pacienta. Po pushi prošly oba běhy CI.

Poučení. Hlášení agenta „hotovo“ ani lokálně zelené jednotkové testy nejsou důkaz. Za hotové beru jen pushnutý commit se zeleným CI, a pushuji průběžně, ne na konci dne. Když lokálně nejde spustit část testů, je to chyba, kterou je potřeba opravit hned, ne přeskočit.

9 Přiznání práce s AI

Nástroje. Claude Code (agent v terminálu) s modely Claude Sonnet 5.5 (začátek projektu, 23. 10. 2026) a Claude Opus 5.5 (od 27. 10. 2026). Commity, na kterých se agent podílel, mají v popisu řádek `Co-Authored-By: Claude …`; týká se to prakticky všech commitů v repozitáři.

Na které části. Agent podle mého zadání napsal kód všech vrstev (controllery, služby, repozitáře, entity), migrace Flyway a syntetická data, webové rozhraní, jednotkové, integrační i prohlížečové testy, Docker a CI konfiguraci, README a tuto dokumentaci včetně ER diagramu. Ode mě pochází volba zadání a technologií, pravidla pro práci agenta (`CLAUDE.md`: vrstvy, žádné SQL v controlleru, nic se nemaže, jen syntetická data) a rozhodnutí k otevřeným bodům zadání (oddíl Rozhodnutí v README), která jsem s agentem procházel a schvaloval.

Jak byl výsledek ověřen.

- Automaticky: `./mvnw verify` s jednotkovými a integračními testy proti skutečné PostgreSQL, test ArchUnit hlídající vrstvy, Playwright E2E testy na desktopu i tabletu a CI při každém pushi (včetně hledání hesel a skenu zranitelností). Pravidla jako nepřekrývání termínů drží i databáze, takže chyba v aplikační logice nemůže poškodit data.
- Ručně: aplikaci jsem proklikal v prohlížeči a s pomocí nahrané prohlídky pro klienta (`e2e/prohlidka/`) prošel všechny funkce. Kód jsem četl po jednotlivých commitech, abych ho uměl vysvětlit a upravit při obhajobě.
- To, že ověřování nebylo bez chyb, popisuje protokol o zacyklení výše.